

Scenario 1: Search Functionality

Context: Act as a Senior QA Engineer testing a B2B E-commerce platform.

Instruction:

[MANDATORY] Generate 5 test cases for the global Search Functionality.

[MANDATORY] Include scenarios for exact SKU matches, partial product name matches, empty inputs, and filter/sort behavior.

[MANDATORY] Include at least one performance/load perspective.

Example: Verify exact product SKU search | High | User is logged in | Enter exact SKU 'AXYZ' in search bar and hit Enter | Only the specific product is returned in under 2 seconds.

Persona: Act as a detail-oriented Test Lead reviewing for edge cases and usability.

Output: Format as a Markdown Table with columns: TCID, Description, Priority, Preconditions, Steps, Expected Result.

Tone: Professional, precise, and analytical.

2. Output Comparison Simulation

Feature	Weak Prompt Output ("Test the search feature.")	ICEPOT Prompt Output
Focus	Very generic (e.g., "Search for a valid item", "Search for an invalid item").	Highly specific to B2B e-commerce (e.g., SKU matches, exact vs. partial).
Structure	Unformatted bulleted list.	Neatly formatted table ready for Jira/Excel import.
Depth	Misses non-functional requirements entirely.	Includes strict performance criteria (e.g., "returned in under 2 seconds").

3. Three Observations on Improvements

1. The ICEPOT prompt forces the AI to understand the context (B2B e-commerce), meaning it tests for things like "SKUs" rather than generic "words."
2. By specifying the exact columns and table output, the generated test cases require zero reformatting before pasting into a test management tool.
3. Forcing specific parameters (empty inputs, filter/sort, performance) ensures the AI doesn't just generate "happy path" UI tests, resulting in a much more resilient test suite.

Scenario 2: File Upload

Context: Act as a Security and Functional QA Specialist for an Enterprise HR Portal where candidates upload their resumes.

Instruction:

[MANDATORY] Generate 10 P0 test cases for the Resume File Upload feature.

[MANDATORY] Validate supported file types (.pdf, .docx) and file size limits (maximum 5MB).

[MANDATORY] Include security test cases

Example: Validate valid PDF upload | High | User is on application page | Upload a 2MB .pdf file
-> Click Submit | File is accepted, progress bar shows 100%, and success message is displayed.

Persona: Review from Security Auditor and QA Lead looking for vulnerabilities.

Output: Table format with columns: TCID, Description, Type (Functional/Security), Steps, Expected Result.

Tone: Authoritative, clear, and security-focused.

2. Output Comparison Simulation

Feature	Weak Prompt Output ("Generate test cases for file upload.")	ICEPOT Prompt Output

Boundaries	Vague boundaries (e.g., "Test a large file").	Precise boundaries (e.g., "Test file exactly 5.01MB").
Security	Rarely mentions malicious files unless prompted.	Explicitly includes injection and executable file rejection tests (.exe, .sh).
Context	Assumes a generic system (e.g., uploading an avatar).	Tailored to an HR portal (e.g., uploading .pdf/.docx resumes).

3. Three Observations on Improvements

1. The weak prompt leaves the definition of "large file" up to the tester. The ICEPOT prompt establishes strict business logic (5MB limit, .pdf/.docx only), resulting in exact test data requirements.
2. The ICEPOT prompt forces a "Shift-Left" testing mentality by incorporating security directly into standard functional testing.
3. Asking the AI to act as a "Rigorous Security Auditor" shifts its language from passive to active, ensuring expected results specify *how* the system should fail.

Scenario 3: API Testing

Context: Act as a Backend QA Automation Engineer using Postman to test a SaaS application's backend.

Instruction:

[MANDATORY] Generate 15 backend API test cases for the User Registration endpoint (**POST** /api/v1/register).

[MANDATORY] Validate HTTP status codes (201 Created, 400 Bad Request, 409 Conflict).

[MANDATORY] Include validation for the JSON request payload and the expected JSON response structure.

Example: Successful User Registration | POST | Payload: valid email, valid password | Expected Status: 201 Created | Expected Response: JSON contains valid user_id

Persona: Embody a strict Backend Architect focusing on robust API contracts and reliable status codes.

Output: Formatted Table with columns: TCID, Scenario, HTTP Method, Payload Detail, Expected Status Code, Expected Response body elements.

Tone: Technical, professional, and developer-oriented.

2. Output Comparison Simulation

Feature	Weak Prompt Output ("Write API test cases for user registration.")	ICEPOT Prompt Output
Technical Accuracy	Reads like UI test cases as API tests (e.g., "Click register").	Highly technical, specifying exact endpoints, HTTP verbs (POST), and JSON schemas.
Validation	Vague assertions (e.g., "Registration successful").	Strict contract assertions (e.g., "Validate 201 Created and JSON contains user_id").
Tool Alignment	Generic steps.	Tailored for immediate scripting into Postman/RestAssured.

3. Three Observations on Improvements

1. The ICEPOT output explicitly maps out HTTP Methods (POST) and exact URI paths, which the weak prompt entirely misses.
2. By explicitly asking for HTTP status codes (201, 400, 409), the AI generates backend-appropriate assertions rather than frontend UI reactions.
3. Structured prompt ensures the AI addresses the JSON payload (both what is sent and what is received), which is the core of backend API testing, making the output immediately usable for writing automated test scripts.